

Taller III
Programación 1
Backtracking

1. Elevadores Inteligentes

Un problema común en muchos edificios es el gran tiempo de espera de los elevadores. Imaginemos que el elevador se encuentra inicialmente en el piso 53, y que es llamado desde los pisos 98, 183, 37, 122, 14, 124, 65 y 67. Necesitamos conocer la dirección de movimiento del elevador (descenso o ascenso). Por ejemplo, si la dirección de movimiento es descendente, se atienden los llamados de los pisos 37 y 14, a medida que se desplaza hacia PB. Luego cambia a dirección ascendente y atiende los llamados de los pisos 65, 67, 98, 122, 124, y 183, a medida que se va desplazando.
Al final, fueron atendidas todas las solicitudes, para lo cual el elevador realizó 236 movimientos. Dados dos pisos i y j, el número de movimientos necesarios para ir de i hasta j (y viceversa) es nummov(i, j) = abs(i−j). Debe notarse que esta planificación no es óptima, y el número de movimientos necesarios para atender todas las solicitudes puede reducirse por medio de otra planificación. Por ejemplo, moviendo del 53 al 37, luego al 14 y después la secuencia
65, 67, 98, 122, 124, y 183, se puede reducir el número total de movimientos del ascensor a 208.

Su trabajo consiste en elaborar un programa que, dada la posición inicial de un elevador y la secuencia de llamadas al mismo, minimice el número total de movimientos necesarios para atender todas las solicitudes.

1.1. Formato de Entrada

La primera línea de la entrada consistiría de un número entero N (1 ≤ N ≤ 1000) que representa el número de casos de prueba. Cada caso de prueba tiene dos l´ıneas: la primera l´ınea contiene tres n´umeros enteros T (1 ≤ T ≤ 200), P0(0 ≤ P0 ≤ T) y P (1 ≤ P ≤ 20) que representan el n´umero total de pisos en el edificio, la posición (piso) inicial del elevador, y el número de pisos que debe atender el elevador, respectivamente. La segunda línea contiene una secuencia de a lo sumo T n´ números, separados entre sí por espacios en blanco, que representa la lista de pisos que debe atender el elevador.

1.2. Formato de Salida

Para cada caso de prueba en la entrada, deber´a escribir la planificaci´on optimal que debe realizar el ascensor para atender la lista de pisos correspondiente y el n´umero m´ınimo de movimientos requeridos, de acuerdo con el formato especificado en el ejemplo.

1.3. Ejemplo de Entrada

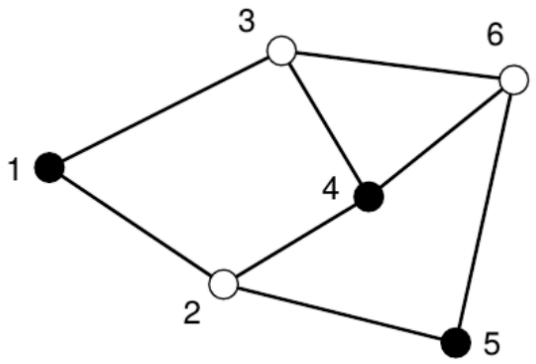
1
200 53 8
98 183 37 122 14 124 65 67

1.4. Ejemplo de Salida

208 Movimientos

2 Píntame el grafo

Debes escribir un programa que intente encontrar una coloración óptima para un grafo dado. Los colores se aplican a los nodos del grafo y solo hay dos colores disponibles: negro y blanco. La coloración se considera óptima si maximiza la cantidad de nodos negros. La coloración está restringida por la regla de que ningún par de nodos conectados puede ser negro.



2.1 Entrada

El grafo viene dado como un conjunto de nodos denotados por números del 1 al n, donde $n \leq 100$, y un conjunto de aristas no dirigidas denotadas por pares de números de nodos (n_1, n_2), siendo $n_1 \neq n_2$. El archivo de entrada contiene m grafos. El número m se da en la primera línea. La primera línea de cada grafo contiene n y k, el número de nodos y el número de aristas respectivamente. Las siguientes líneas contienen las aristas dadas por un par de números de nodos, separados por un espacio.

2.2 Salida

La salida debe consistir en 2m líneas, dos líneas para cada grafo encontrado en el archivo de entrada. La primera línea debe contener el número máximo de nodos que pueden ser coloreados de negro en el grafo. La segunda línea debe contener una coloración óptima posible. Esta se da mediante la lista de nodos negros, separados por espacios en blanco.

2.3 Ejemplo de entrada

1

6 8

1 2

1 3

2 4

2 5

3 4

3 6

4 6

5 6

2.4 Ejemplo de Salida

3
1 4 5

3. Juan el esquizofrénico

Un hombre solía jugar dominó con algunos amigos en un pequeño pueblo. Algunas familias han abandonado el pueblo y en la actualidad los únicos residentes en el pueblo son el hombre y su esposa. Este hombre quiere continuar jugando dominó, pero su esposa no le gusta jugar. Él ha inventado un juego para jugar solo. Se eligen dos piezas y se colocan en los dos extremos de una fila con un número (n) de espacios. Después, se eligen otras piezas (m) para llenar los espacios. El número de piezas es mayor o igual que el número de espacios ($m \geq n$), y el número de piezas es menor o igual que 14 ($m \leq 14$). Los espacios se llenan colocando una pieza en cada espacio, según las reglas del dominó: el número de puntos adyacentes en dos dominós diferentes debe coincidir. Las piezas con valores repetidos se colocan de la misma manera que las demás, y no en ángulos rectos. El problema consiste en diseñar un programa que, dado un número de espacios (n), un número de piezas (m), las dos piezas iniciales (i1, i2) y (d1, d2), y las m piezas (p1, q1), (p2, q2), ..., (pm, qm), decida si es posible llenar los n espacios entre las dos piezas iniciales usando las m piezas y con las reglas del dominó. Por ejemplo, con n = 3, m = 4, piezas iniciales (0,1) y (3,4), y piezas (2,1), (5,6), (2,2) y (3,2), la respuesta es 'YES', porque existe una solución: (0,1), (1,2), (2,2), (2,3), (3,4). Con n = 2, m = 4, piezas en los extremos (0,1) y (3,4), y (1,4), (4,4), (3,2) y (5,6), la respuesta es 'NO'.

3.1 entrada

La entrada consistirá en una serie de problemas, donde cada problema se describe en una serie de líneas: en la primera línea se indica el número de espacios (n), en la segunda línea el número de piezas (m) utilizadas para llenar los espacios, en la siguiente línea la pieza que se colocará a la izquierda, con los dos valores en la pieza separados por un espacio y de la misma manera que aparecen los números en la fila, en las siguientes líneas la pieza que se colocará a la derecha, con los dos valores en la pieza separados por un espacio y de la misma manera que aparecen los números en la fila, y las otras m piezas aparecen en líneas consecutivas, una en cada línea, con los dos valores separados por un espacio. Los diferentes problemas aparecen sucesivamente en la entrada sin separación, y la entrada termina cuando aparece '0' como número de espacios.

3.2 Salida

Para cada problema en la entrada se escribe una línea, con 'YES' si el problema tiene solución, y 'NO' si no tiene solución.

Entrada	**Salida**
3 4 0 1 3 4 2 1 5 6 2 2 3 2 2 4 0 1	SI NO

3 4 1 4 4 4 3 2 5 6 0	

Ejercicio 4 Pizza para el salon

Eres responsable de ordenar una pizza grande para ti y tus amigos de clase. Cada uno de ellos te ha dicho qué quiere en la pizza y qué no quiere; por supuesto, todos entienden que como solo va a haber una pizza, es probable que nadie tenga todas sus peticiones satisfechas. ¿Puedes ordenar una pizza que satisfaga al menos una petición de todos tus amigos? La pizzería del cafetín imaginario ofrece los siguientes ingredientes para pizza; puedes incluir o omitir cualquiera de ellos en la pizza:

```
A Anchoas
B Aceitunas Negras
C Tocino Canadiense
D Ajo Picado
E Queso Extra
F Brócoli Fresco
G Pimientos Verdes
H Jamón
I Salchicha Italiana
J Pimientos Jalapeño
K Kielbasa
L Carne Molida Magra
M Champiñones
N Queso Feta Sin Grasa
O Cebollas
P Pepperoni
```

Tus amigos te proporcionan una línea de texto que describe sus preferencias de pizza. Por ejemplo, la línea +O-H+P; revela que alguien aceptará una pizza con cebolla, o sin jamón, o con pepperoni, y la línea -E-I-D+A+J; indica que otra persona aceptará una pizza que omita queso extra, o salchicha italiana, o ajo picado, o que incluya anchoas o pimientos jalapeños.

4.1 Entrada

La entrada consiste en una serie de restricciones de pizza. Una restricción de pizza es una lista de 1 a 12 listas de restricciones de ingredientes, cada una en una línea por sí misma, seguida por un punto en una línea por sí misma. Una lista de restricciones de ingredientes es una serie de solicitudes de ingredientes terminadas por un solo punto y coma. Una solicitud de ingrediente es un carácter de signo (+/-) y luego una letra mayúscula de la 'A' a la 'P'.

4.2 Salida

Para cada restricción de pizza, proporciona una descripción de una pizza que la satisfaga. Una descripción es la cadena 'Toppings: ' en las columnas 1 a través de 10 y luego una serie de letras, en orden alfabético, que listan los ingredientes en la pizza. Así, una pizza con cebolla, anchoas, brócoli fresco y tocino canadiense sería descrita por:

Toppings: ACFO

Si no se puede encontrar ninguna combinación de ingredientes que satisfaga al menos una petición de cada persona, su programa debe imprimir la cadena “No pizza can satisfy these requests”.

Entrada	Salida
+A+B+C+D-E-F-G-H; -A-B+C+D-E-F+G+H; -A+B-C+D-E+F-G+H; . +A+B+C+D; +E+F+F+H; +A+B-G; +O+J-F; +H+I+C; +P; +O+M+L; +M-L+P; . +A+B+C+D; +E+F+F+H; +A+B-G; +P-O; +O+J-F; +H+I+C; +P; +O; +O+M+L; -O-P; +M-L+P; .	Toppings: Toppings: CELP No pizza can satisfy these requests.